S. B. JAIN INSTITUTE OF TECHNOLOGY, MANAGEMENT & RESEARCH, NAGPUR.
Session 2025-26 [Even]

Practical No. 1 (Pre-Lab)

Aim: Understanding Google Colab and **Python Libraries** for Machine Learning. [**NumPy, Pandas, Matplotlib, Seaborn,** Scikit-learn].

Name of Student : **Abhishek Yadav**

Roll No. : **CS23071**

Semester/Year : **VI-III**

Academic Session : **2025-2026**

Date of Performance : _____

Date of Submission : _____

AIM: Understanding Google Colab and Python Libraries for Machine Learning. [NumPy, Pandas, Matplotlib, Seaborn, Scikit-learn].

OBJECTIVE/EXPECTED LEARNING OUTCOME:

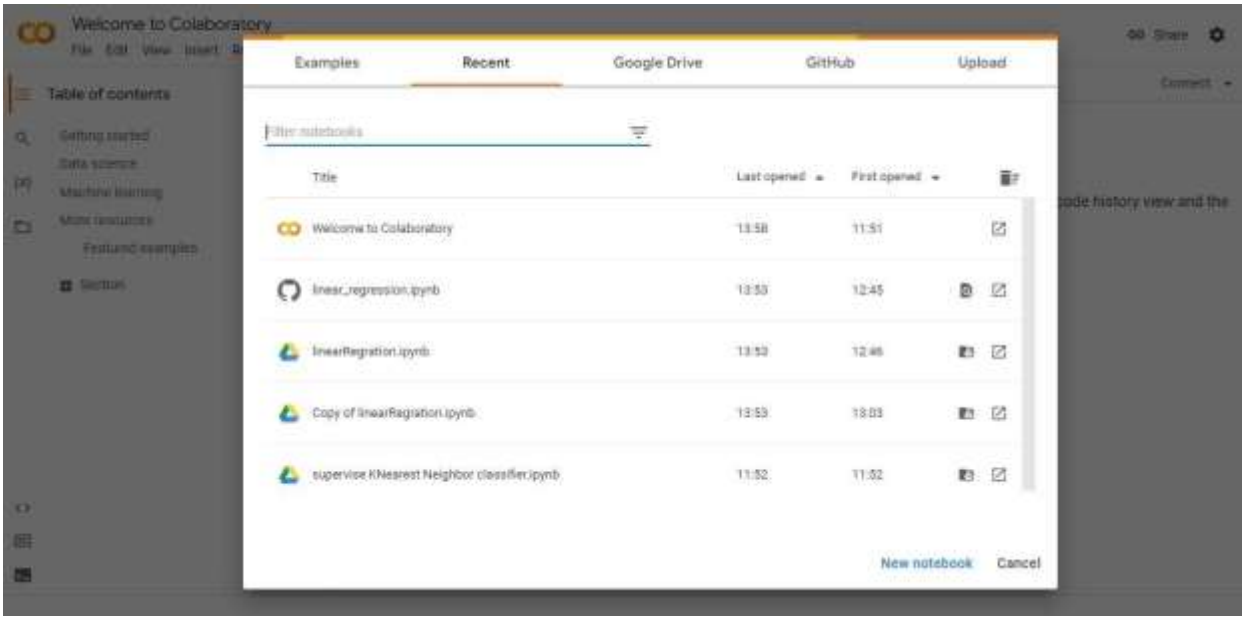
The objectives and expected learning outcome of this practical are:

- Able to work in the environment of Google Colab
- Run Python codes on Google Colab.
- Understand Python language fundamentals, including basic syntax, variables, and types
- Create and manipulate different data structures Python can handle, such as Lists, Dictionaries, sets, and Tuples.
- Use Python control and loops statements.
- Define functions in Python and reuse them.
- Use built-in functions in Python such as ZIP, MAP, and lambda
- Learn about various date time manipulation solutions in Python ● Read, write and manipulate files in Python.
- Use basic and advanced data analysis tools such as NumPy, Pandas, Matplotlib, Seaborn, Scikit-learn.

THEORY:

Prerequisite: How to use Google Colab Google provides Jupyter Notebook like interface to run Python code on online Virtual Machines. In this article, we will see how to run simple Python code on Google Colab.

Step #1: Open https://colab.research.google.com/

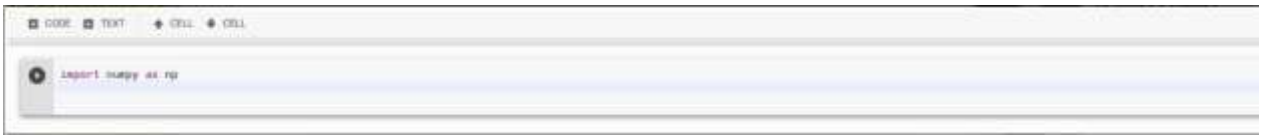


Step #2: Click on the New Notbook

Step #3: Give the name of file with the extension .ipynb

Step #4: Select typing code in the code cell. Import all necessary file

\



Step #5: To add new cell, click on Insert->Code Cell

1. NumPy (Numerical Python)

NumPy is the core library for numerical computation in Python and forms the backbone of most ML and data-science libraries.

Key Features:

1. **N-Dimensional Array Object (ndarray)**
   Provides a powerful ndarray object.
   - Faster and more memory-efficient than Python lists. Supports 1D, 2D, and multi-dimensional arrays.
2. **Vectorized Operations**
   - Performs operations on entire arrays without explicit loops. Significantly improves performance and execution speed.
3. **Mathematical & Statistical Functions**
   Built-in functions for:
   - Mean, median, variance, standard deviation
   - Min, max, sum, cumulative sum

   Used extensively in data preprocessing.
4. **Linear Algebra Support**
   - Matrix multiplication (dot)
   - Transpose, inverse, determinant
   - Eigenvalues and eigenvectors
   - Used in ML algorithms like Linear Regression, PCA, SVM.
5. **Random Number Generation**
   Functions to generate:
   - Uniform and normal distributions
   - Random integers and floats
   - Useful for simulations and ML initialization.
6. **Broadcasting**
   - Allows operations between arrays of different shapes. Reduces memory usage and simplifies code.
7. **Integration & Performance** Written in C → high performance.
   - Easily integrates with Pandas, Matplotlib, Scikit-learn.

2. Pandas

Pandas is a data manipulation and preprocessing library designed to work with structured data.

Key Features:

1. **Data Structures (Series & DataFrame)**
   - **Series**: One-dimensional labeled array.
   - **DataFrame**: Two-dimensional tabular data structure.
2. **Data Loading & Exporting** Reads data from:

- CSV, Excel, SQL, JSON, HTML

Exports data to multiple formats.

3. **Data Cleaning & Preprocessing**
   - Handling missing values (fillna, dropna)
   - Removing duplicates Data type conversion
   - Renaming columns
4. **Data Selection & Indexing**
   - Label-based indexing (loc)
   - Integer-based indexing (iloc)
   - Boolean filtering
5. **Grouping & Aggregation**
   - groupby() for aggregation and summarization Statistical operations like mean, sum, count.
6. **Categorical Data Handling**
   - Encoding categorical variables
   - One-hot encoding
   - Label encoding

3. Matplotlib

Matplotlib is the foundation visualization library in Python.

Key Features:

1. Variety of Plot Types

- Line plots
- Bar charts
- Scatter plots
- Histograms
- Pie charts **2. High Customization** Control over:
  - Colors, markers, line

styles
  - Axes, labels, ticks
  - Titles and legends

3. **Figure & Subplot Control**
   - Multiple plots in one figure
   - Flexible layout management
4. **Integration with NumPy & Pandas**
   - Direct plotting from arrays and DataFrames.
5. **Export Options**
   - Save figures as PNG, JPG, PDF, SVG.

4. Seaborn

Seaborn is a statistical visualization library built on top of Matplotlib.

Key Features:

1. High-Level Interface

- Fewer lines of code to create complex plots. ▯ Automatic handling of Pandas DataFrames.

2. Statistical Plots ▯ Box plots

- Violin plots
- Pair plots
- Distribution plots

3. Relationship Visualization

- Visualizes relationships between variables. ▯ Supports categorical and numerical data.

4. Correlation Analysis

- Heatmaps for correlation matrices.
- Annotated values for better interpretation.

5. Aesthetic Themes

- Built-in themes for professional-looking plots. ▯ Consistent color palettes.

5. Scikit-learn

Scikit-learn is a machine learning library that provides tools for model building, evaluation, and deployment. **Key Features:**

1. Supervised Learning Algorithms

▯ Regression:

○ Linear

Regression ○ Ridge, Lasso ▯

Classification:

○ Logistic

Regression

○ KNN ○

SVM

○ Decision Trees

2. Unsupervised Learning

Algorithms ▯ Clustering: ○

K-Means

○ Hierarchical

clustering ▯ Dimensionality

Reduction: ○ PCA

○ LDA

3. Data Preprocessing Tools

- Feature scaling (StandardScaler, MinMaxScaler)
- Encoding categorical data
- Handling missing values

4. Model Selection & Validation

- Train-test split
- Cross-validation
- Grid search and hyperparameter tuning

5. Model Evaluation Metrics Regression metrics: MAE, MSE, RMSE, R^2

- Classification metrics: Accuracy, Precision, Recall, F1-score

6. Pipeline & Workflow Management ML pipelines for clean, reproducible workflows.

7. Model Persistence

- Save and load trained models using joblib.

PROGRAM CODE:

OUTPUT (SCREENSHOT):

Abhishek Yadav_(CS23071)

Pre-Lab: Understanding Google Colab and Python Libraries for Machine Learning,[NumPy, Pandas,Matplotlib,Seaborn,Sckit-learn]

```
import numpy as np
#create arrays
a = np.array([1, 2, 3, 4])
b = np.zeros((2,3))
c = np.ones((2,3))
d = np.random.rand(2,3)
print(a)
print(b)
print(c)
print(d)
```

```
[1 2 3 4]
[[0. 0. 0.]
 [0. 0. 0.]]
[[1. 1. 1.]
 [1. 1. 1.]]
[[0.65256791 0.6634882  0.5033757 ]
 [0.49200901 0.62903725 0.9802838 ]]
```

```
# array operations
print(a.shape)
print(a.mean)
print(a.std())
print(np.sum(a))
print(np.max(a))
```

```
(4,)
<built-in method mean of numpy.ndarray object at 0x7ce80ac7b4b0>
1.118033988749895
10
4
```

```
# matrix opertions
x = np.array([[1, 2], [3, 4]])
y = np.array([[5, 6], [7, 8]])
print(np.dot(x, y))
print(np.transpose(x))
```

```
[[19 22]
 [43 50]]
[[1 3]
 [2 4]]
```

```
import pandas as pd
# create DataFrame
data = {
    'Age': [25, 30, 45],
    'Salary': [30000, 40000, 50000],
    'Department':['IT', 'HR', 'Finance']
}
df = pd.DataFrame(data)
print(df.describe())
```

```
...                Age    Salary
count    3.000000       3.0
mean    33.333333   40000.0
std     10.408330   10000.0
min     25.000000   30000.0
25%     27.500000   35000.0
50%     30.000000   40000.0
75%     37.500000   45000.0
max     45.000000   50000.0
```

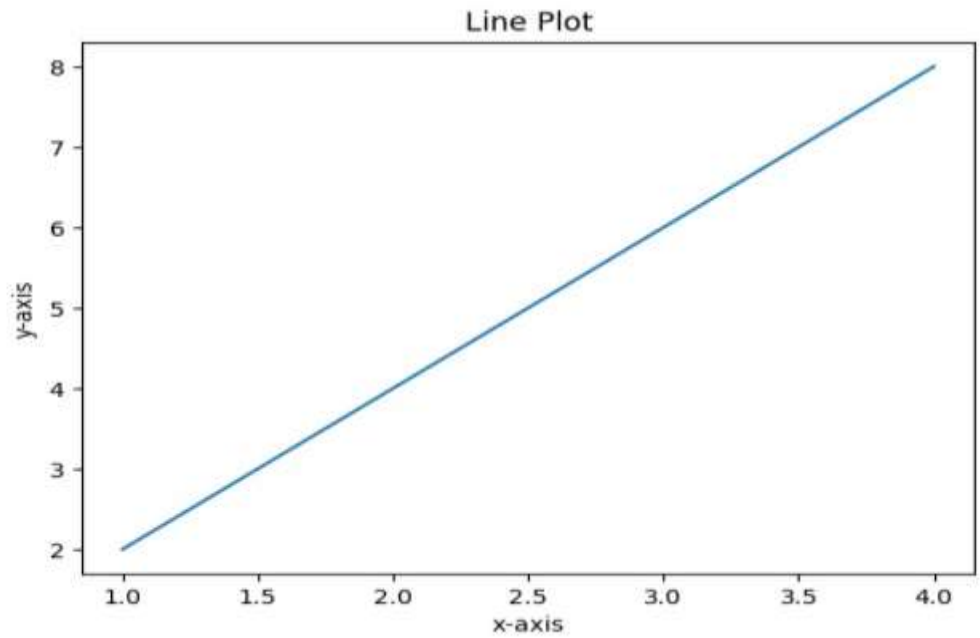
```python
import pandas as pd
# encoding categorial data
data = {
    'Age': [25, 30, 45],
    'Salary': [30000, 40000, 50000],
    'Department':['IT', 'HR', 'Finance']
}
df_encoded = pd.get_dummies(df, drop_first=True)
print(df_encoded)
```

```
     sepal_length  sepal_width  petal_length  petal_width  species_versicolor  \
0             5.1          3.5           1.4          0.2               False
1             4.9          3.0           1.4          0.2               False
2             4.7          3.2           1.3          0.2               False
3             4.6          3.1           1.5          0.2               False
4             5.0          3.6           1.4          0.2               False
..            ...          ...           ...          ...                 ...
145           6.7          3.0           5.2          2.3               False
146           6.3          2.5           5.0          1.9               False
147           6.5          3.0           5.2          2.0               False
148           6.2          3.4           5.4          2.3               False
149           5.9          3.0           5.1          1.8               False

     species_virginica
0                False
1                False
2                False
3                False
4                False
..                 ...
145               True
146               True
147               True
148               True
149               True

[150 rows x 6 columns]
```

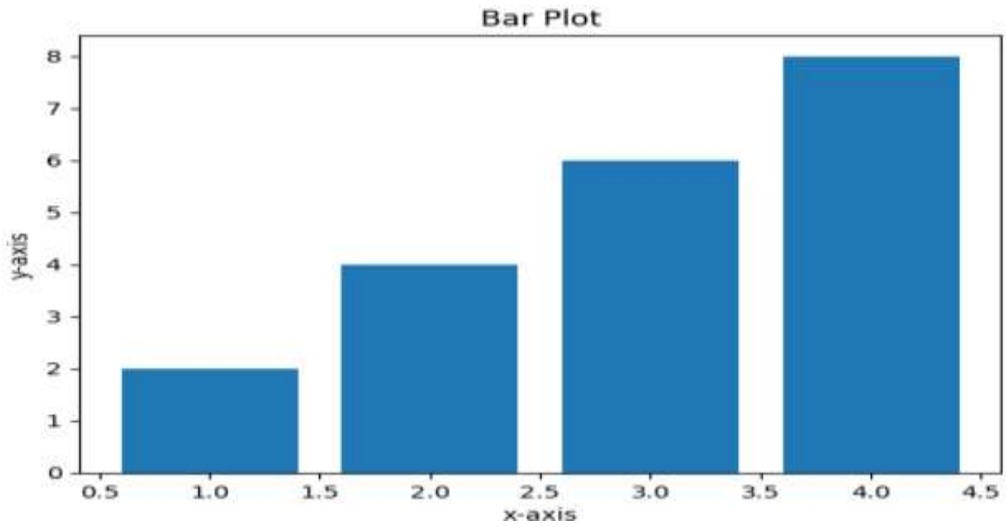
```python
df_encoded = pd.get_dummies(df, drop_first=True)
print(df_encoded)
```

```
     sepal_length  sepal_width  petal_length  petal_width  species_versicolor
0             5.1          3.5           1.4          0.2               False
1             4.9          3.0           1.4          0.2               False
2             4.7          3.2           1.3          0.2               False
3             4.6          3.1           1.5          0.2               False
4             5.0          3.6           1.4          0.2               False
..            ...          ...           ...          ...                 ...
145           6.7          3.0           5.2          2.3               False
146           6.3          2.5           5.0          1.9               False
147           6.5          3.0           5.2          2.0               False
148           6.2          3.4           5.4          2.3               False
149           5.9          3.0           5.1          1.8               False

     species_virginica
0                False
1                False
2                False
3                False
4                False
..                 ...
145               True
146               True
147               True
148               True
149               True

[150 rows x 6 columns]
```

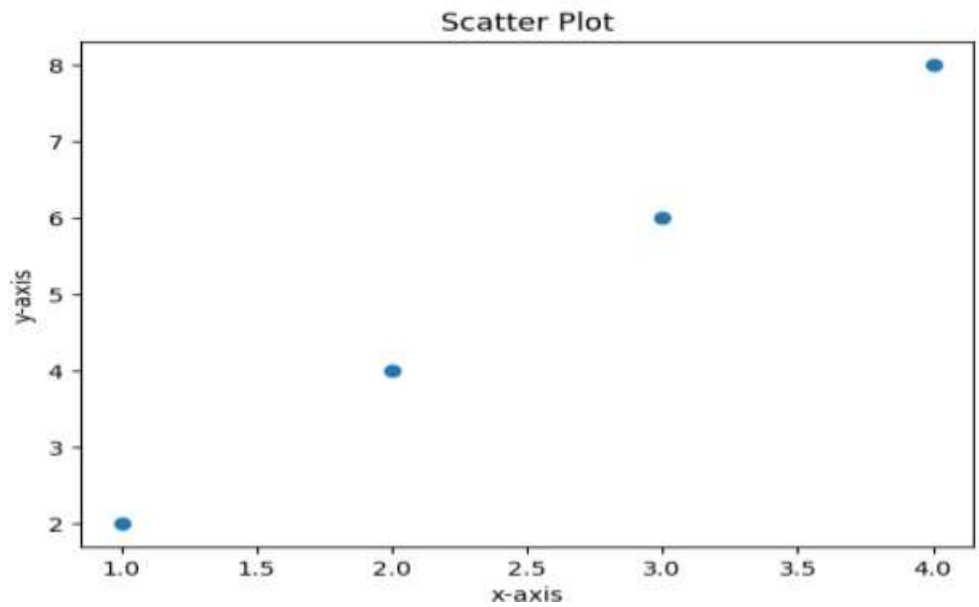
```python
import matplotlib.pyplot as plt
x = [1, 2, 3, 4]
y = [2, 4, 6, 8]
plt.plot(x, y)
plt.xlabel('x-axis')
plt.ylabel('y-axis')
plt.title('Line Plot')
plt.show()
```



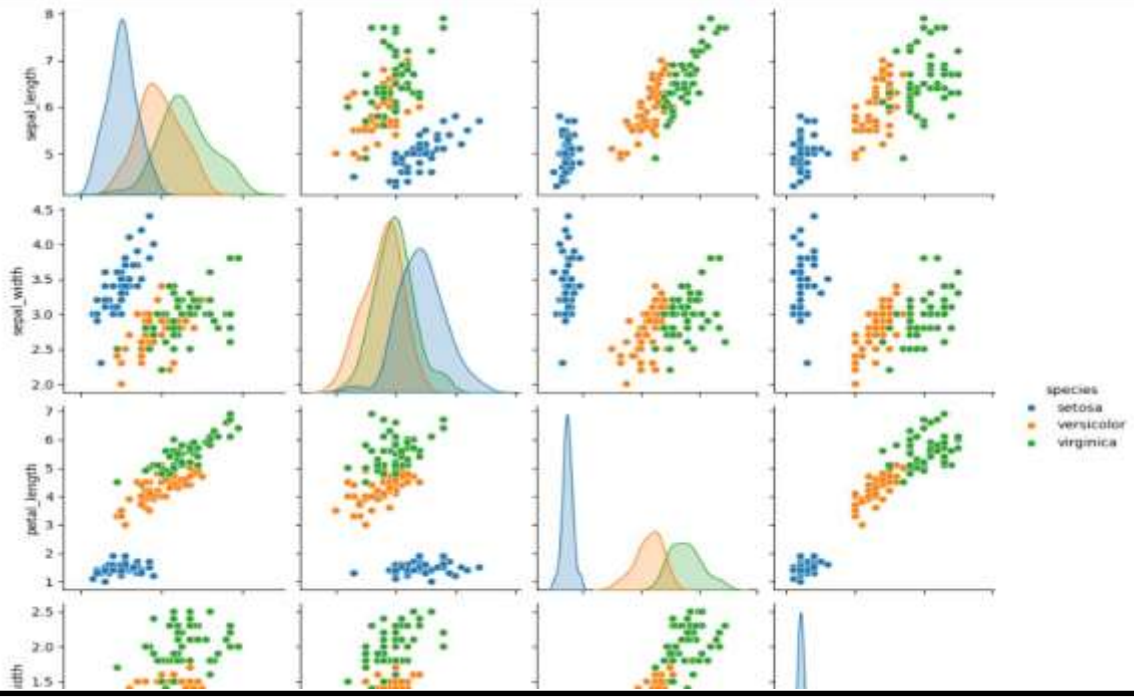

```python
# bar plot
plt.bar(x, y)
plt.xlabel('x-axis')
plt.ylabel('y-axis')
plt.title('Bar Plot')
plt.show()
```



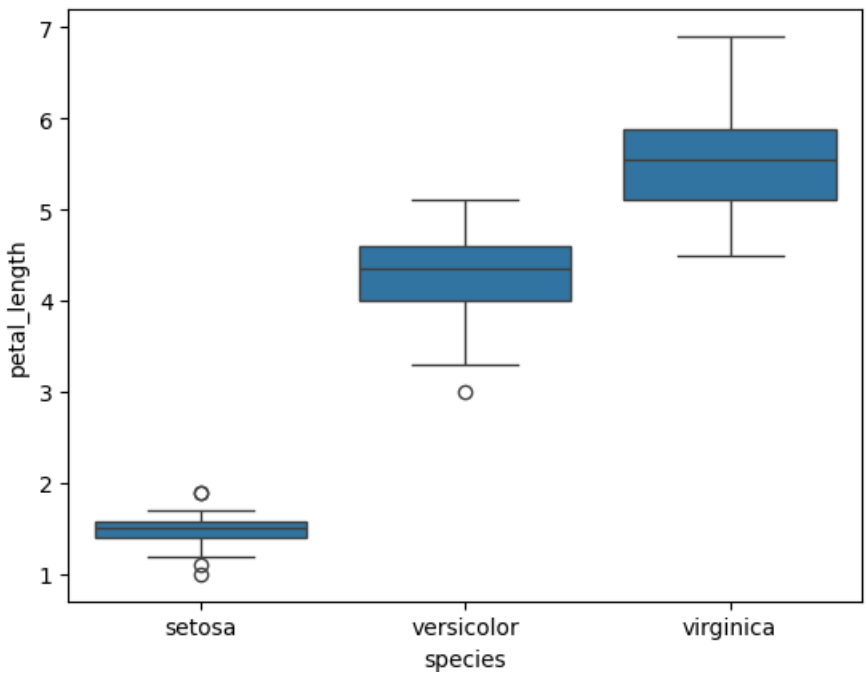
```
# scatter plot
plt.scatter(x, y)
plt.xlabel('x-axis')
plt.ylabel('y-axis')
plt.title('Scatter Plot')
plt.show()
```




```
import seaborn as sns
import matplotlib.pyplot as plt
# load sample dataset
df = sns.load_dataset('iris')
# create pair plot
sns.pairplot(df, hue='species')
plt.show()
```



```python
# Box plot
sns.boxplot(x='species', y='petal_length', data=df)
plt.show()
```



CONCLUSION: Hence we understand the concept of Google Colab and Python Libraries for Machine Learning. [NumPy, Pandas, Matplotlib, Seaborn, Scikit-learn].

DISCUSSION AND VIVA VOCE:

1. Why is NumPy preferred over Python lists in machine learning projects?
2. How does NumPy vectorization improve performance in large-scale data processing?
3. In real projects, why is Pandas used before applying machine learning models?
4. How do you handle missing and inconsistent data in Pandas for production datasets?
5. Why is data visualization important before model training, and which Matplotlib plots are commonly used in industry?
6. When would you prefer Seaborn over Matplotlib in an analytics or ML project?
7. How does feature scaling in Scikit-learn impact model performance in real-world applications?
8. Why is train-test split or cross-validation critical in industry ML systems?

9. How do Scikit-learn pipelines help in building production-ready ML solutions?
10. How do you evaluate whether a trained ML model is acceptable for deployment in industry?

REFERENCE:

- https://towardsdatascience.com/practical-machine-learning-tutorial-part-1-data-exploratoryanalysis-c13d39b8f33b
- https://pubs.acs.org/doi/10.1021/acsomega.2c00362
- https://www.interviewbit.com/blog/pandasumpy/#:~:text=Pandas%20is%20mostly%20used%20for,easy%20to%20apply%20mathematical%20functions.&text=Pandas%20library%20works%20well%20for,heterogeneous%20types%20of%20data%20simultaneously.